

Приёмы программирования на языке JAVA

Лабораторная работа №5.

Разработка многопоточного приложения

Общая постановка задачи.

Разработать приложение для параллельной обработки текстов на основе многопоточной модели.

Требования к результату

1. Приложение должно быть реализовано как автономное (desktop application) и однопользовательское с графическим интерфейсом.
2. Приложение должно обрабатывать текстовые файлы в стандартных кодировках: ASCII, UTF-8 или UTF-16.
3. Приложение должно обеспечивать выполнение следующих функций:
 - 3.1. Обработку локального текстового файла любого размера (в дальнейшем - исходный файл);
 - 3.2. Создание файла отчёта с результатами обработки исходного файла (в дальнейшем - файл отчёта);
 - 3.3. Параллельную обработку исходного файла в соответствии с набором заданных алгоритмов;
 - 3.4. Замену текста в исходном файле на текст, полученный в результате его обработки;
 - 3.5. Занесение в файл отчёта результатов выполнения задач.
4. Алгоритмы обработки исходного текста должны основываться на реализации функционального интерфейса Modifier, исходный код которого приведён в приложении 1:
 - 4.1. Первый параметр метода modify интерфейса Modifier

```
String modify(String text, String template, Mode mode)  
throws TextException
```

содержит ссылку на текущий фрагмент исходного текста, второй – дополнительную строку, содержание и порядок использования которой зависит от третьего параметра, а последний параметр определяет алгоритм, который нужно применить к заданному фрагменту исходного текста;

- 4.2. Алгоритм Mode.COMPRESS должен удалять из исходного текста все идущие подряд символы из заданного набора, заменяя их на один соответствующий символ. Например, три подряд идущих пробела должны быть заменены на один пробел или два подряд идущих символа окончания строки - на один символ конца строки. Список

повторяющихся символов, подлежащих такой обработке, задаётся параметром `template` метода `modify()` в виде регулярного выражения. Метод должен вернуть изменённый текст;

- 4.3. Алгоритм `Mode.COUNT` должен подсчитывать число вхождений в исходный текст строки, заданной параметром `template` метода `modify()`. В этом случае метод должен возвращать строку, содержащую заданное значение параметра `template` и число его вхождений в исходный текст;
- 4.4. Алгоритм `Mode.DELETE` должен удалять из исходного текста все фрагменты, которые описываются регулярным выражением, заданным параметром `template` метода `modify()`. Метод возвращает изменённый текст;
- 4.5. Алгоритм `Mode.FIND` должен извлечь из исходного текста все предложения, которые содержат текст, описываемый регулярным выражением `template`. Предложения должны извлекаться полностью без каких-либо изъятий.
5. Изменённый текст, полученный в результате выполнения алгоритмов `Mode.COMPRESS` и `Mode.DELETE`, должен заменить текст в исходном файле.
6. Результаты выполнения алгоритмов `Mode.COUNT` и `Mode.FIND` должны быть записаны в один файл отчёта.
7. Каждый алгоритм обработки исходного текста должен выполняться **параллельно и независимо в отдельном потоке(!)**, каждый из которых должен быть экземпляром подкласса абстрактного класса `ATextModifier` (см. приложение 2).

Приложения

Приложение 1. Исходный код интерфейса `Modifier`

```
package threads;

/**
 * Функциональный интерфейс, определяющий метод анализа исходного
 * текста.
 *
 * @author Ю.Д.Заковряшин.
 */
@FunctionalInterface
public interface Modifier {

    /**
     * Перечисление определяет алгоритмы обработки исходного текста.
     */
    public static enum Mode {
        COMPRESS("Сжатие исходного текста."),
        COUNT("Подсчёт вхождений в исходном тексте заданного
фрагмента текста."),
        DELETE("Удаление заданного фрагмента из исходного текста"),
        FIND("Извлечение из исходного текста всех упоминаний
фрагмента, заданного регулярным выражением.") .
    }
```

```

    /**
     * Описание алгоритма, который выполняет реализация метода.
     */
    private final String description;

    Mode(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}

/**
 * Метод выполняет заданный алгоритм обработки исходного текста.
 *
 * @param text исходный текст, подлежащий обработке.
 * @param template строка с дополнительным текстом, необходимым
для
    * обработки исходного текста, например, она может содержать
фрагмент
    * текста, который должен быть удалён из исходного текста в
режиме
    * Mode.DELETE.
    * @param mode описывает алгоритм, который выполняется в
реализации метода.
    * @return изменённый текст.
    * @throws TextException выбрасывается в случае возникновения
ошибки, связанной с обработкой текста.
 */
String modify(String text, String template, Mode mode) throws
TextException;
}

```

Приложение 2. Исходный код класса ATextModifier

```

package threads;

import java.io.File;
import java.util.concurrent.Semaphore;

/**
 * Класс определяет отдельный поток для параллельной обработки
текстовых файлов.
 *
 * @author Ю.Д.Заковряшин
 */
public abstract class ATextModifier implements Runnable {

    /**
     * Семафор, который используется для сигнала о блокировке
     * исходного файла в ситуации, когда данный поток производит
     * запись в исходный файл.
     */
    private static Semaphore writeInput;

    /**
     * Семафор, который используется для сигнала о блокировке
     * файла отчёта в ситуации, когда данный поток производит

```

```

    * запись в файл отчёта.
    */    private static Semaphore writeReport;
/**
    * Ссылка на файл с исходным текстом.
    */
private static File input;
/**
    * Ссылка на файл отчёта о результатах обработки исходного файла.
    */
private static File report;
}

```

Рекомендации по работе

В реализации должно быть учтено следующее:

1. Исходный файл может быть очень большим, поэтому приложение должно обрабатывать исходный файл по частям, каждая из которых не должна превышать 8 Кб;
2. Обрабатываемый фрагмент исходного файла должна содержать семантически законченный текст, т.е. выделяя очередной фрагмент нельзя разрывать предложения;
3. Объекты типа Semaphore следует использовать для учёта блокировки файлов в процессе записи их содержимого разными потоками.